

Inter-Service Communication Flow

Architecture Overview

The system follows a microservices architecture consisting of:

- API Gateway
- Authentication Microservice
- Leave Manager Microservice
- Notifications Microservice
- RabbitMQ Message Broker
- Independent Databases per Service

Communication Patterns

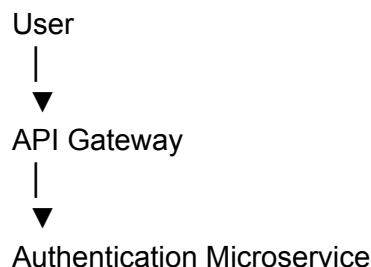
Pattern	Usage
Synchronous HTTP	API Gateway ↔ Microservices
Service-to-Service HTTP	Leave-Manager → AuthService (JWKS Validation)
Asynchronous Messaging	Leave-Manager → RabbitMQ → Notifications Service

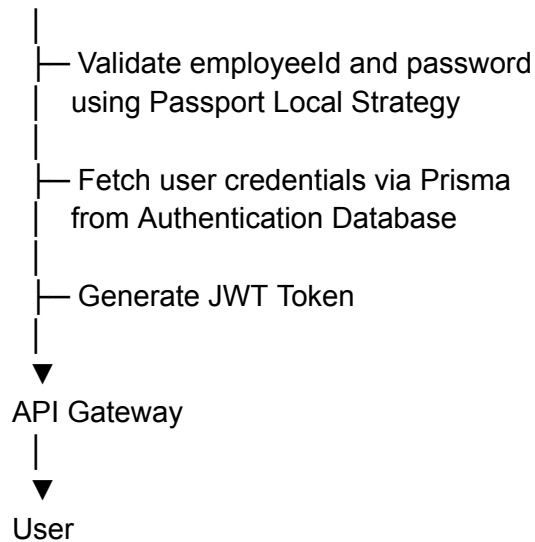
1. Authentication Flow (Login)

Endpoint

POST /auth/login

Flow





Internal Processing

1. User submits `employeeId` and `password`.
 2. API Gateway forwards request to Authentication Microservice.
 3. Passport Local Strategy validates credentials.
 4. Prisma fetches user record from Authentication Database.
 5. Authentication Service generates JWT containing:
 - `employeeId`
 - `role`
 - `expiration time`
 6. JWT is returned to the user via API Gateway.
-

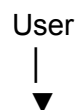
2. Authorization Flow (JWT Validation)

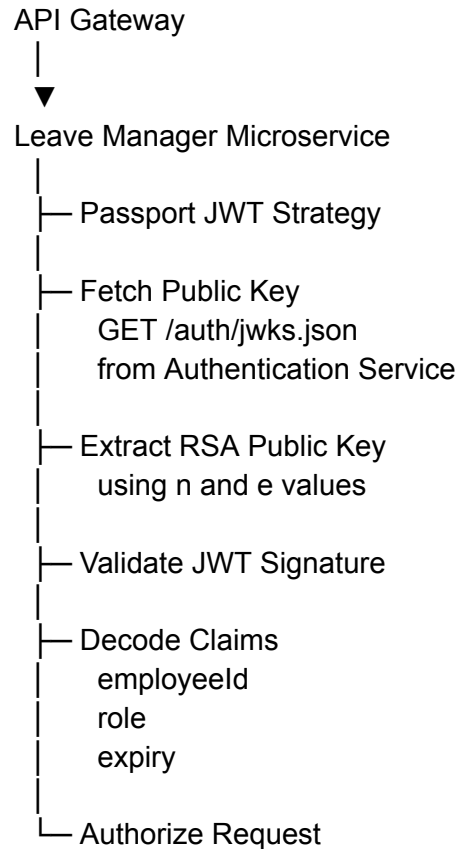
All protected endpoints follow the same authorization process.

Example Endpoint

GET `/leave-manager/view-balance`

Flow





Internal Processing

1. User sends JWT in Authorization header.
2. API Gateway forwards request to Leave Manager Service.
3. Leave Manager JWT Strategy retrieves the public signing key from:

/auth/jwks.json

4. Authentication Service returns JWKS containing:

```
{
  "keys": [
    {
      "n": "...",
      "e": "..."
    }
  ]
}
```

5. JWT signature is validated.
6. Payload is decoded.
7. Role-based authorization is applied.
8. Request proceeds to business logic.

This approach ensures:

- Centralized key management
 - Secure key rotation
 - No public key duplication across services
-

3. View Own Leave Balance

Endpoint

GET /leave-manager/view-balance

Flow

User



API Gateway



Leave Manager Microservice



— Passport JWT Strategy



— Fetch Public Key
GET /auth/jwks.json
from Authentication Service



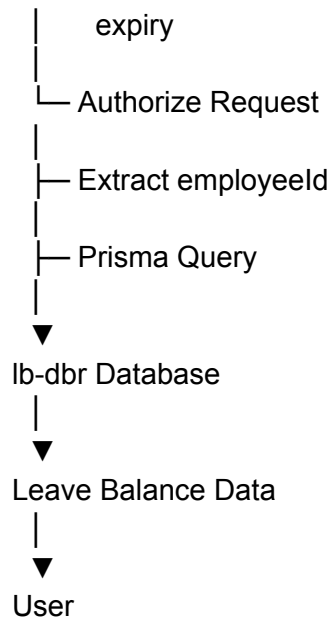
— Extract RSA Public Key
using n and e values



— Validate JWT Signature



— Decode Claims
employeeId
role



Internal Processing

1. JWT is validated.
 2. Employee ID is extracted from the token.
 3. Prisma fetches balance.
 4. Response is returned to user.
-

4. View Employee Leave Balance (Manager)

Endpoint

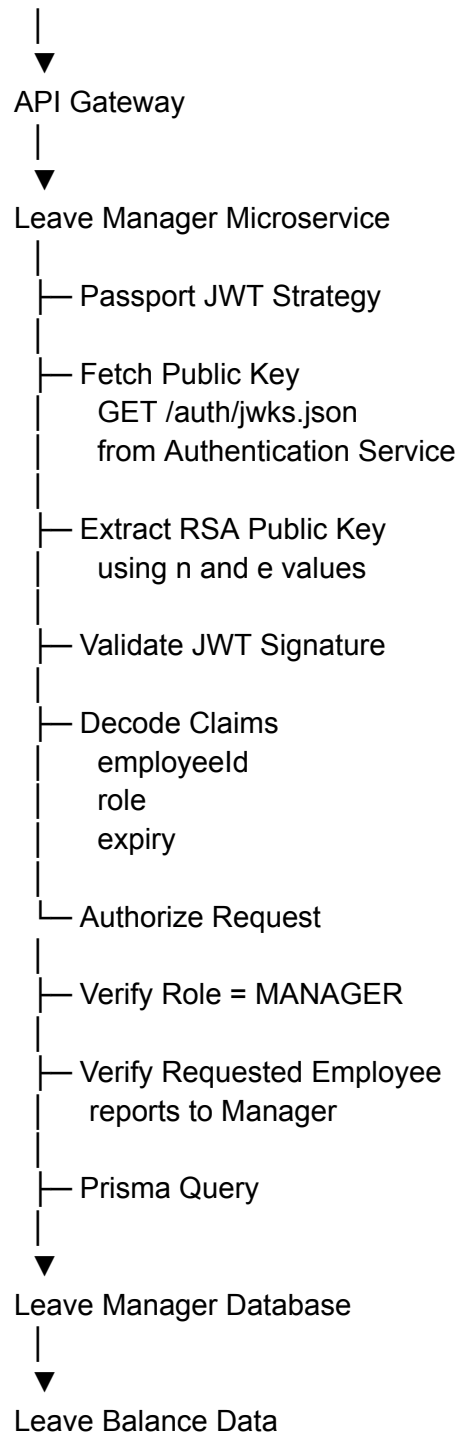
GET /leave-manager/view-balance/employee?empId={employeeId}

Access Control

Manager Only

Flow

Manager



Internal Processing

1. JWT validation occurs.
2. User role must be Manager.
3. Employee must belong to the manager's reporting hierarchy.

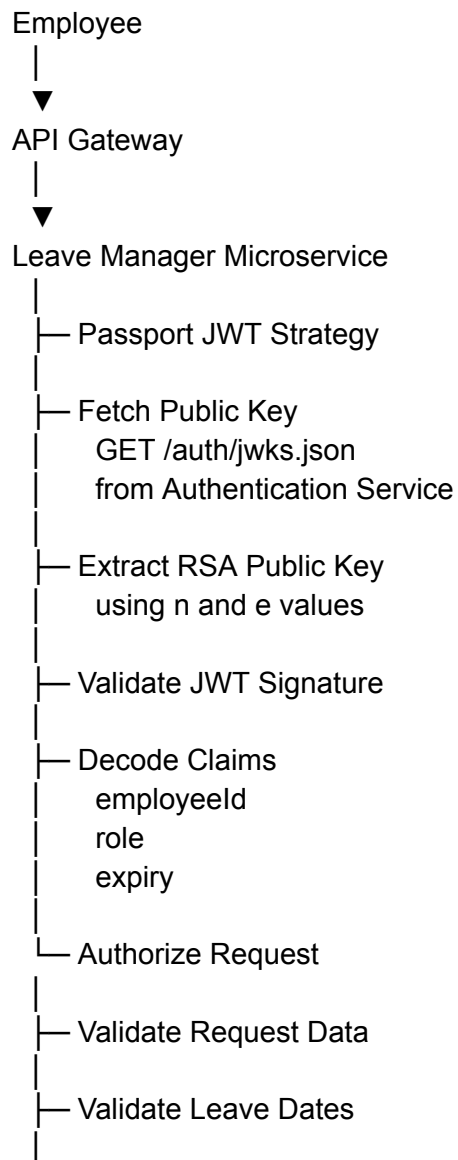
4. Prisma retrieves leave balance.
 5. Response is returned.
-

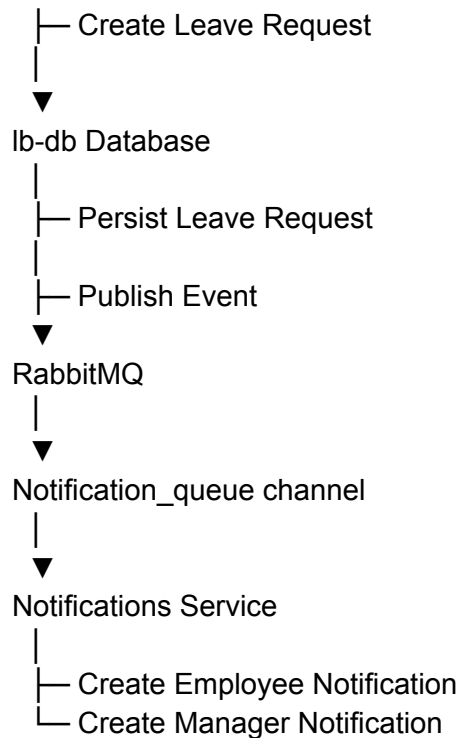
5. Apply Leave

Endpoint

POST /leave-manager/leaves/apply

Flow





Internal Processing

1. JWT is validated.
2. Leave request payload is validated.
3. Date validations are performed.
4. Leave request record is created.
5. Leave balance is **not deducted** at this stage.
6. Leave Manager publishes an event to RabbitMQ.
7. Notifications Service consumes the event.
8. Notifications are generated for:
 - Employee
 - Reporting Manager

6. Approve Leave

Endpoint

POST /leave-manager/leaves/employees/approve

Access Control

Manager Only

Flow

Manager



API Gateway



Leave Manager Microservice



— Passport JWT Strategy

— Fetch Public Key
GET /auth/jwks.json
from Authentication Service

— Extract RSA Public Key
using n and e values

— Validate JWT Signature

— Decode Claims
employeeId
role
expiry

— Authorize Request

— Verify Manager Access

— Validate Request

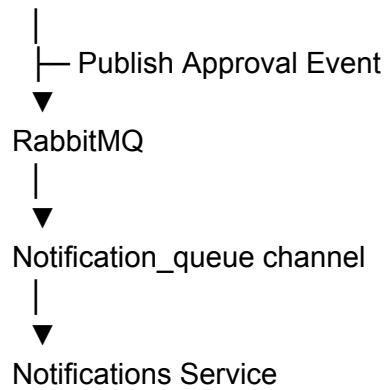
— Update Leave Request

— Calculate Business Days

— Deduct Leave Balance



Leave Manager Database



Internal Processing

1. Manager authorization is verified.
 2. Leave request status is updated to Approved.
 3. **Business days are calculated.**
 4. **Saturdays and Sundays are excluded.**
 5. **Leave balance is deducted.**
 6. Approval event is published.
 7. Notification Service creates employee notification.
-

7. Reject Leave

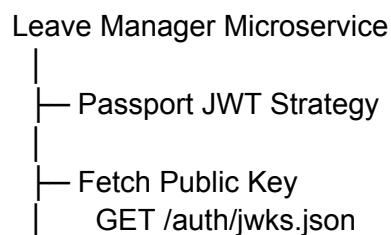
Endpoint

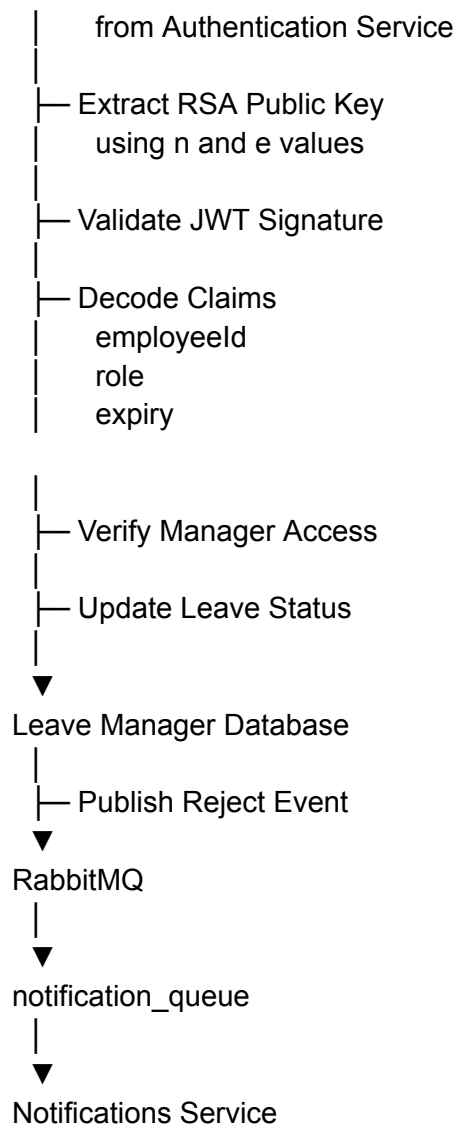
POST /leave-manager/leaves/employees/reject

Access Control

Manager Only

Flow





Internal Processing

1. JWT validation succeeds.
2. Manager authorization is verified.
3. Leave request status is updated to Rejected.
4. Reject event is published.
5. Notifications Service creates employee notification.

8. Cancel Leave

Endpoint

POST /leave-manager/leaves/employees/cancel

Access Control

Employee or Manager

Business Rules

- User can cancel only their own leave request.
- Users cannot cancel leave requests of other employees.
- Used when a leave was submitted by mistake.

Flow

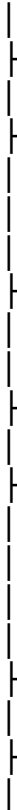
User



API Gateway



Leave Manager Microservice



— Passport JWT Strategy

— Fetch Public Key
GET /auth/jwks.json
from Authentication Service

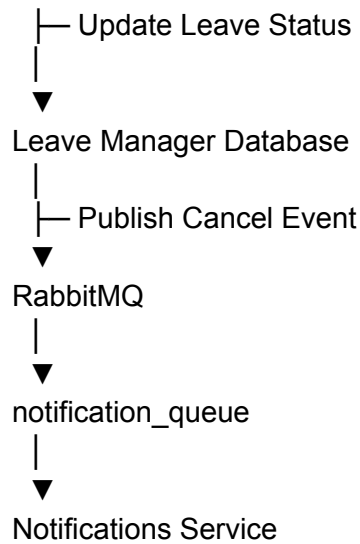
— Extract RSA Public Key
using n and e values

— Validate JWT Signature

— Decode Claims
employeeId
role
expiry

— Verify Ownership

— Validate Request



Internal Processing

1. JWT is validated.
2. Employee ownership of leave request is verified.
3. Leave request is marked as Cancelled.
4. Cancellation event is published.
5. Notifications Service creates employee notification.

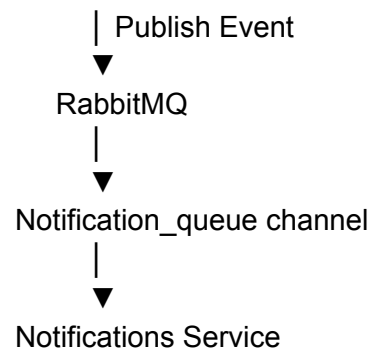
Event-Driven Communication Summary

Producer Service	RabbitMQ Queue	Consumer Service	Event Type
Leave Manager	notification_queue	Notifications Service	Leave Applied
Leave Manager	notification_queue	Notifications Service	Leave Approved
Leave Manager	notification_queue	Notifications Service	Leave Rejected
Leave Manager	notification_queue	Notifications Service	Leave Cancelled

Messaging Pattern

Leave Manager Service

|



All notification events are implemented using a **fire-and-forget asynchronous messaging pattern**, ensuring that leave operations are completed without waiting for notification processing. This improves service decoupling, scalability, and fault tolerance.